IC TRAINING CENTER VIET NAM
FUNDAMENTAL IC DESIGN AND VERIFICATION COURSE

FINAL PROJECT

TIMER IP DESIGN SPECIFICATION (ADVANCED LEVEL)

Student: Trinh Minh Quan
Class: IC41
Instructor: Tran Minh Duc

CONTENTS

1. Overview
   1.1. Introduction
   1.2. Main Features
   1.3. Block Diagram
   1.4. Interface Signals
2. Register Specification
   2.1. Register Summary
   2.2. Timer Control Register (TCR)
   2.3. Timer Data Register 0 (TDR0)
   2.4. Timer Data Register 1 (TDR1)
   2.5. Timer Compare Register 0 (TCMP0)
   2.6. Timer Compare Register 1 (TCMP1)
   2.7. Timer Interrupt Enable Register (TIER)
   2.8. Timer Interrupt Status Register (TISR)
   2.9. Timer Halt Control Status Register (THCSR)
3. Functional Description
   3.1. APB Slave Interface
   3.2. Register Module
      3.2.1. Write Path and Specific Register Implementations
      3.2.2. Read Path and RAZ/WI Implementation
   3.3. Counter Control Module
      3.3.1. Logic Implementation
      3.3.2. Timing and Behavior
   3.4. Counter Module
      3.4.1. Logic Implementation and Operation Priority
      3.4.2. Advanced Features Timing
   3.5. Interrupt Logic
      3.5.1. Logic Implementation and Clear Priority
      3.5.2. Interrupt Lifecycle Timing

Trinh Minh Quan

4. Revision History

LIST OF TABLES

Table 1.1: Interface signals of Timer IP (timer_top).

Table 2.1: Register Summary.

Table 2.2: Timer Control Register (TCR).

Table 2.3: Timer Data Register 0 (TDR0).

Table 2.4: Timer Data Register 1 (TDR1).

Table 2.5: Timer Compare Register 0 (TCMP0).

Table 2.6: Timer Compare Register 1 (TCMP1).

Table 2.7: Timer Interrupt Enable Register (TIER).

Table 2.8: Timer Interrupt Status Register (TISR).

Table 2.9: Timer Halt Control Status Register (THCSR).

LIST OF FIGURES

Figure 1.1: Top-level Block Diagram of Timer IP.

Figure 3.1: APB Slave Timing Waveform (Wait State, Byte Access & Error Response).

Figure 3.2: APB Slave Logic Diagram.

Figure 3.3: TCR Logic Diagram and Error Generation.

Figure 3.4: TDR0 and TDR1 Write Path Routing.

Figure 3.5: TCMP0 Register Logic Diagram Demonstrating Byte Access.

Figure 3.6: THCSR Register Logic Diagram.

Figure 3.7: Counter Control Logic Diagram.

Figure 3.8: Counter Control Modes Timing Waveform.

Figure 3.9: Counter Logic Diagram Illustrating Strict Prioritization.

Figure 3.10: Hardware Auto-Clear and Halted Mode Timing Waveform.

Figure 3.11: Interrupt Registers (TIER/TISR) Logic Diagram (Generation and RW1C Clearance).

Figure 3.12: Interrupt Generation and RW1C Clearance Timing Waveform.

1. Overview

1.1. Introduction

A timer is an essential peripheral module in any System-on-Chip (SoC) design, primarily utilized to generate accurate timing intervals, hardware delays, pulse-width modulation (PWM) timebases, and event triggers. In this project, the Timer IP is customized and developed based on the Core-Local Interruptor (CLINT) module from the industrial RISC-V architecture. It functions as a highly configurable 64-bit count-up timer that generates maskable hardware level-triggered interrupts when the running counter matches a user-defined 64-bit compare threshold. This makes the IP critical for operating system tick generation, precise task scheduling, and reliable hardware synchronization.

1.2. Main Features

Bus Interface & System Parameters:

- Operates as an AMBA APB slave configured via a 12-bit address space (4 KB addressable range).
- Supports standard 32-bit APB transfers as well as individual byte access via the 4-bit write strobe signal (tim_pstrb[3:0]).
- Implements a 1-cycle wait state (via tim_pready) for every read and write transfer to improve internal timing closure.
- Supports hardware error handling (tim_pslverr) for prohibited accesses: writing an invalid divisor value (div_val > 4'h8) or modifying div_en/div_val while the timer is actively counting (timer_en = 1). When an error occurs, write data is ignored and register state is preserved.
- All read/write accesses to reserved addresses or reserved bit fields strictly follow RAZ/WI (Read-As-Zero, Write-Ignored) semantics.
- Uses an active-low asynchronous reset signal (sys_rst_n).

Counter Features:

- 64-bit free-running count-up timer split into two 32-bit registers (TDR0 and TDR1).
- Supports two counting modes: Default Mode (counts at system clock frequency) and Control Mode (counting speed divided from 1 up to 256 based on TCR.div_val when TCR.div_en = 1).

- Continues counting seamlessly when an interrupt condition or counter overflow occurs.
- Hardware automatically clears the counter registers (TDR0 and TDR1) to their initial reset values (32'h0000_0000) when timer_en transitions from High to Low (H -> L). Normal counting resumes when timer_en transitions from Low to High (L -> H).

Debug & Halted Mode:

- Supports halting (freezing) the counter when the system operates in debug mode.
- The counter halts when both the external dbg_mode input is High (1) and software sets the halt request bit (THCSR.halt_req = 1).
- Provides a read-only halt acknowledge status flag (THCSR.halt_ack = 1) indicating that the halt request has been accepted. The counter resumes counting from its halted state while preserving the exact division period once THCSR.halt_req is cleared to 0.

Interrupt Features:

- Generates a hardware, maskable, level-triggered interrupt (tim_int).
- The interrupt output (tim_int) is asserted High when interrupt generation is enabled (TIER.int_en = 1) and the 64-bit counter value matches the 64-bit compare threshold ({TCMP1, TCMP0}).
- Once asserted, the interrupt pending status (TISR.int_st) remains latched at 1 until explicitly cleared by software writing 1 to TISR.int_st (RW1C). Disabling TIER.int_en masks tim_int to 0 without clearing TISR.int_st.

1.3. Block Diagram

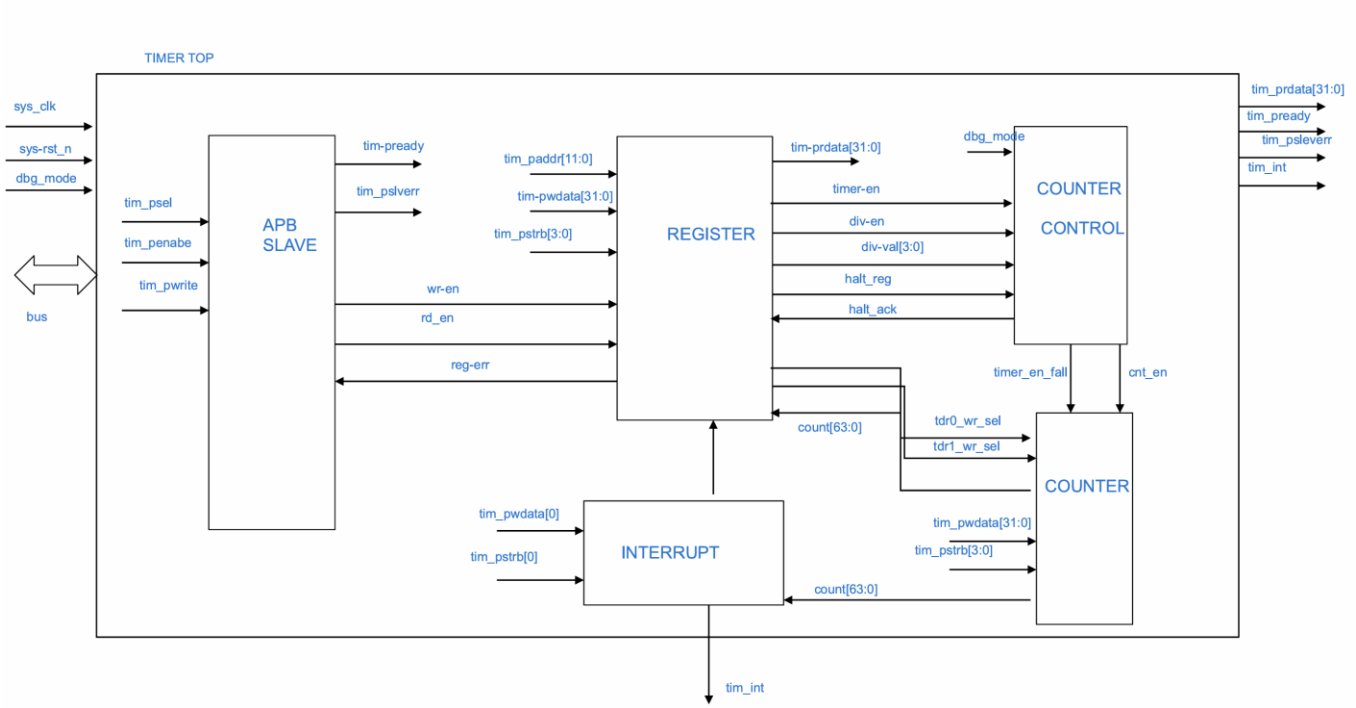


Figure 1.1: Top-level Block Diagram of Timer IP (timer_top).

Sub-Block Descriptions:

- **APB SLAVE:** Acts as the protocol interface between the external APB Master and the Timer IP. It handles APB handshaking, inserts a 1-cycle wait state to generate synchronous write/read strobes (wr_en, rd_en), drives tim_pready, and forwards internal register error flags to tim_pslverr.
- **REGISTER:** Functions as the address decoder, static configuration storage (TCR, TCMP0, TCMP1, TIER, TISR, THCSR), and read data multiplexer. It enforces RAZ/WI rules on reserved spaces, validates write configurations to assert reg_err on illegal accesses, and routes write strobes and byte enables to the Counter module.
- **COUNTER CONTROL:** Manages the counting rate and debug halt control. Using an 8-bit internal prescaler (int_cnt[7:0]), it evaluates timer_en, div_en, and div_val to generate the synchronous count enable pulse (cnt_en). It also asserts halt_ack when (dbg_mode & halt_req) is active, freezes both the prescaler and cnt_en during halt, and detects the falling edge of timer_en (timer_en_fall).

- **COUNTER:** Houses the 64-bit counter composed of TDR0 (lower 32 bits) and TDR1 (upper 32 bits). It enforces a strict operation hierarchy: hardware auto-clear on timer_en_fall (highest priority), APB byte-level write preset via wstrb[3:0] (medium priority), and 64-bit increment on cnt_en (lowest priority).

1.4. Interface Signals

The top-level module is named timer_top. All external interface signals are detailed in Table 1.1:

Table 1.1: Interface signals of Timer IP (timer_top).

Signal Name	Width	Direction	Description
sys_clk	1	Input	System clock driving all sequential logic inside the Timer IP.
sys_rst_n	1	Input	Active-low asynchronous reset (0: reset state, 1: normal operation).
tim_psel	1	Input	APB slave select signal asserted by the APB Master.
tim_pwrite	1	Input	APB transfer direction control (1: Write transfer, 0: Read transfer).
tim_penable	1	Input	APB enable signal indicating the Access phase of an APB transfer.
tim_paddr[11:0]	12	Input	APB 12-bit byte address bus.
tim_pwdata[31:0]	32	Input	APB 32-bit write data bus.
tim_prdata[31:0]	32	Output	APB 32-bit read data bus.

tim_pstrb[3:0]	4	Input	APB write byte strobe signal supporting individual byte access (tim_pstrb[i] corresponds to tim_pwdata[(8*i+7):(8*i)]).
tim_pready	1	Output	APB ready signal used to insert a 1-cycle wait state.
tim_pslverr	1	Output	APB slave error response signal asserted on prohibited register accesses.
tim_int	1	Output	Maskable, level-triggered timer interrupt output.
dbg_mode	1	Input	System debug mode indicator (1: debug mode active). Assumed stable while timer_en is High.

2. Register Specification

2.1. Register Summary

Table 2.1: Register Summary.

Offset	Abbreviation	Register Name	Reset Value	Access
0x000	TCR	Timer Control Register	32'h0000_0100	RW
0x004	TDR0	Timer Data Register 0 (Lower 32-bit)	32'h0000_0000	RW
0x008	TDR1	Timer Data Register 1 (Upper 32-bit)	32'h0000_0000	RW
0x00C	TCMP0	Timer Compare Register 0 (Lower 32-bit)	32'hFFFF_FFFF	RW

0x010	TCMP1	Timer Compare Register 1 (Upper 32-bit)	32'hFFFF_FFFF	RW
0x014	TIER	Timer Interrupt Enable Register	32'h0000_0000	RW
0x018	TISR	Timer Interrupt Status Register	32'h0000_0000	RW1C
0x01C	THCSR	Timer Halt Control Status Register	32'h0000_0000	RW / RO
Others	Reserved	Reserved Address Space	32'h0000_0000	RAZ/WI

2.2. Timer Control Register (TCR)

Offset address: 0x000 | Reset value: 0x0000_0100

31	**30**	**29**	**28**	**27**	**26**	**25**	**24**	**23**	**22**	**21**	**20**	**19**	**18**	**17**	**16**
Reserved															
ro															
15	**14**	**13**	**12**	**11**	**10**	**9**	**8**	**7**	**6**	**5**	**4**	**3**	**2**	**1**	**0**
Reserved				**DIV_VAL[3:0]**				**Reserved**						**DIV_EN**	**TIM_EN**
ro				*rw*				*ro*						*rw*	*rw*

Table 2.2: Timer Control Register (TCR).

Bit	Name	Type	Default Value	Description

31:12	Reserved	RO	20'h00000	Reserved. Read as zero, write ignored (RAZ/WI).
11:8	div_val	RW	4'b0001	Counter control mode divisor setting: • 4'b0000: Counting speed is not divided (div by 1). • 4'b0001: Counting speed is divided by 2 (default). • 4'b0010: Counting speed is divided by 4. • 4'b0011: Counting speed is divided by 8. • 4'b0100: Counting speed is divided by 16. • 4'b0101: Counting speed is divided by 32. • 4'b0110: Counting speed is divided by 64. • 4'b0111: Counting speed is divided by 128. • 4'b1000: Counting speed is divided by 256. • Others (4'b1001 - 4'b1111): Reserved, prohibited settings. [Advanced]: Writing a prohibited value (> 4'h8) or attempting to modify div_val while timer_en is High (1) triggers an APB error response (tim_pslverr = 1) and leaves the register unchanged.

7:2	Reserved	RO	6'b000000	Reserved. Read as zero, write ignored (RAZ/WI).
1	div_en	RW	1'b0	Counter control mode enable: • 0: Disabled. Counter counts at normal system clock speed. • 1: Enabled. Counter speed is controlled by div_val. [Advanced]: Attempting to modify div_en while timer_en is High (1) triggers an APB error response (tim_pslverr = 1) and leaves the register unchanged.
0	timer_en	RW	1'b0	Timer enable: • 0: Disabled. Counter is stopped. • 1: Enabled. Counter starts counting. [Advanced]: When timer_en transitions from High to Low (H -> L), hardware automatically clears TDR0 and TDR1 to their initial reset values (32'h0000_0000).

2.3. Timer Data Register 0 (TDR0)

Offset address: 0x004 | Reset value: 0x0000_0000

31	**30**	**29**	**28**	**27**	**26**	**25**	**24**	**23**	**22**	**21**	**20**	**19**	**18**	**17**	**16**
TDR0[31:16]															
rw															
15	**14**	**13**	**12**	**11**	**10**	**9**	**8**	**7**	**6**	**5**	**4**	**3**	**2**	**1**	**0**

TDR0[15:0]
rw

Table 2.3: Timer Data Register 0 (TDR0).

Bit	Name	Type	Default Value	Description
31:0	TDR0	RW	32'h0000_0000	Lower 32 bits of the 64-bit counter. [Advanced]: Automatically cleared to its initial value (32'h0000_0000) by hardware when timer_en transitions from High to Low (H -> L).

2.4. Timer Data Register 1 (TDR1)

Offset address: 0x008 | Reset value: 0x0000_0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
TDR1[31:16]															
rw															
15	**14**	**13**	**12**	**11**	**10**	**9**	**8**	**7**	**6**	**5**	**4**	**3**	**2**	**1**	**0**
TDR1[15:0]															
rw															

Table 2.4: Timer Data Register 1 (TDR1).

Bit	Name	Type	Default Value	Description
31:0	TDR1	RW	32'h0000_0000	Upper 32 bits of the 64-bit counter.

				[Advanced]: Automatically cleared to its initial value (32'h0000_0000) by hardware when timer_en transitions from High to Low (H -> L).

2.5. Timer Compare Register 0 (TCMP0)

Offset address: 0x00C | Reset value: 0xFFFF_FFFF

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
TCMP0[31:16]															
rw															
15	**14**	**13**	**12**	**11**	**10**	**9**	**8**	**7**	**6**	**5**	**4**	**3**	**2**	**1**	**0**
TCMP0[15:0]															
rw															

Table 2.5: Timer Compare Register 0 (TCMP0).

Bit	Name	Type	Default Value	Description
31:0	TCMP0	RW	32'hFFFF_FFFF	Lower 32 bits of the 64-bit compare value. The interrupt pending bit (TISR.int_st) is asserted when the 64-bit counter value equals {TCMP1, TCMP0}.

2.6. Timer Compare Register 1 (TCMP1)

Offset address: 0x010 | Reset value: 0xFFFF_FFFF

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
TCMP1[31:16]															
rw															
15	**14**	**13**	**12**	**11**	**10**	**9**	**8**	**7**	**6**	**5**	**4**	**3**	**2**	**1**	**0**
TCMP1[15:0]															
rw															

Table 2.6: Timer Compare Register 1 (TCMP1).

Bit	Name	Type	Default Value	Description
31:0	TCMP1	RW	32'hFFFF_FFFF	Upper 32 bits of the 64-bit compare value. The interrupt pending bit (TISR.int_st) is asserted when the 64-bit counter value equals {TCMP1, TCMP0}.

2.7. Timer Interrupt Enable Register (TIER)

Offset address: 0x014 | Reset value: 0x0000_0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															
ro															
15	**14**	**13**	**12**	**11**	**10**	**9**	**8**	**7**	**6**	**5**	**4**	**3**	**2**	**1**	**0**
Reserved															**INT_EN**
ro															*rw*

Table 2.7: Timer Interrupt Enable Register (TIER).

Bit	Name	Type	Default Value	Description
31:1	Reserved	RO	31'h0	Reserved. Read as zero, write ignored (RAZ/WI).
0	int_en	RW	1'b0	Timer interrupt enable: • 0: Disabled. External interrupt output (tim_int) is masked to 0. • 1: Enabled. External interrupt output reflects the interrupt pending bit (TISR.int_st). Note: Clearing this bit to 0 while an interrupt is asserted masks tim_int to 0 immediately without clearing TISR.int_st.

2.8. Timer Interrupt Status Register (TISR)

Offset address: 0x018 | Reset value: 0x0000_0000

31	**30**	**29**	**28**	**27**	**26**	**25**	**24**	**23**	**22**	**21**	**20**	**19**	**18**	**17**	**16**
Reserved															
ro															
15	**14**	**13**	**12**	**11**	**10**	**9**	**8**	**7**	**6**	**5**	**4**	**3**	**2**	**1**	**0**
Reserved															**INT_ST**
ro															*rw1c*

Table 2.8: Timer Interrupt Status Register (TISR).

Bit	Name	Type	Default Value	Description
31:1	Reserved	RO	31'h0	Reserved. Read as zero, write ignored (RAZ/WI).
0	int_st	RW1C	1'b0	Timer interrupt trigger condition status bit (pending bit): • 0: Interrupt trigger condition has not occurred. • 1: Interrupt trigger condition has occurred (counter matched compare value). Clearance (RW1C): Writing 1 when this bit is 1 clears it to 0. Writing 0 when this bit is 1 has no effect. Writing any value when this bit is 0 has no effect. Note 1: When the compare match occurs, the counter continues counting normally. Note 2: Software clearance (writing 1) has the highest priority.

2.9. Timer Halt Control Status Register (THCSR)

Offset address: 0x01C | Reset value: 0x0000_0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Reserved															

ro															
15	**14**	**13**	**12**	**11**	**10**	**9**	**8**	**7**	**6**	**5**	**4**	**3**	**2**	**1**	**0**
Reserved														**HALT_ACK**	**HALT_REQ**
ro														*ro*	*rw*

Table 2.9: Timer Halt Control Status Register (THCSR).

Bit	Name	Type	Default Value	Description
31:2	Reserved	RO	30'h0	Reserved. Read as zero, write ignored (RAZ/WI).
1	halt_ack	RO	1'b0	Timer halt acknowledge status (Advanced Level): • 0: Timer is not halted (running or idle). • 1: Timer is currently halted in debug mode. Note: The timer accepts a halt request only when the external debug_mode (dbg_mode) signal is High (1).
0	halt_req	RW	1'b0	Timer halt request control (Advanced Level): • 0: No halt request (normal counting operation). • 1: Request the timer counter to halt (effective only when dbg_mode = 1).

3. Functional Description

3.1. APB Slave Interface

The Timer IP integrates an APB Slave module to communicate with the system APB Master. This module decodes APB bus transactions, inserts a mandatory 1-cycle wait state, and generates synchronous internal control pulses (wr_en, rd_en) for the Register module. Key architectural properties include:

- Address and Data Width: Operates with a 12-bit byte address bus (tim_paddr[11:0]) and 32-bit data buses (tim_pwdata[31:0], tim_prdata[31:0]).
- Byte Access Support: Routes the 4-bit write strobe signal (tim_pstrb[3:0] -> wstrb[3:0]) to selectively update individual byte lanes during write transactions.
- 1-Cycle Wait State Generation: Delays tim_pready by one system clock cycle after tim_psel and tim_penable are both asserted.
- Error Response Handling: Asserts tim_pslverr High alongside tim_pready when the Register module flags an illegal access (reg_err = 1).

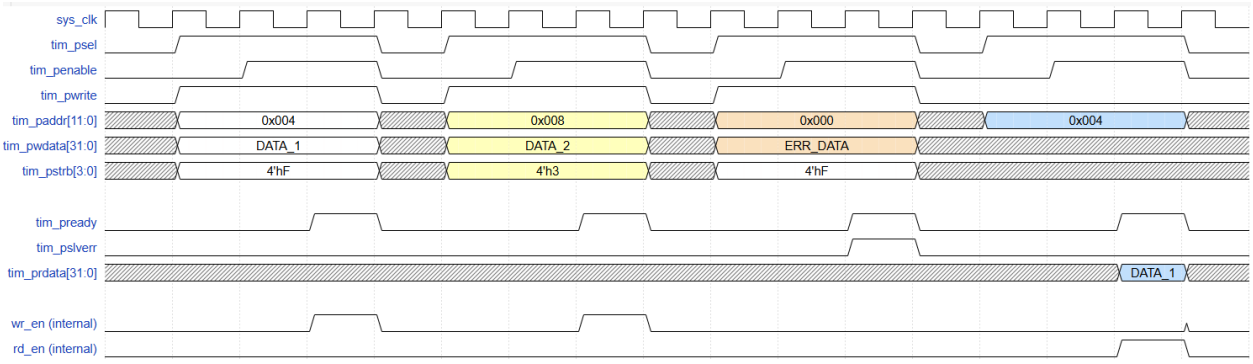


Figure 3.1: APB Slave Timing Waveform (Wait State, Byte Access & Error Response).

Waveform Analysis: As shown in Figure 3.1, every APB transfer incorporates a 1-cycle wait state. During a write transfer, tim_pready is asserted one clock cycle after tim_penable goes High, coinciding with the internal wr_en pulse. If an illegal configuration write is attempted on TCR, reg_err asserts tim_pslverr High during the completion cycle while masking the register write enable so invalid data is discarded. Similarly, during a read transfer, rd_en and tim_pready assert one cycle after tim_penable goes High, providing valid read data on tim_prdata.

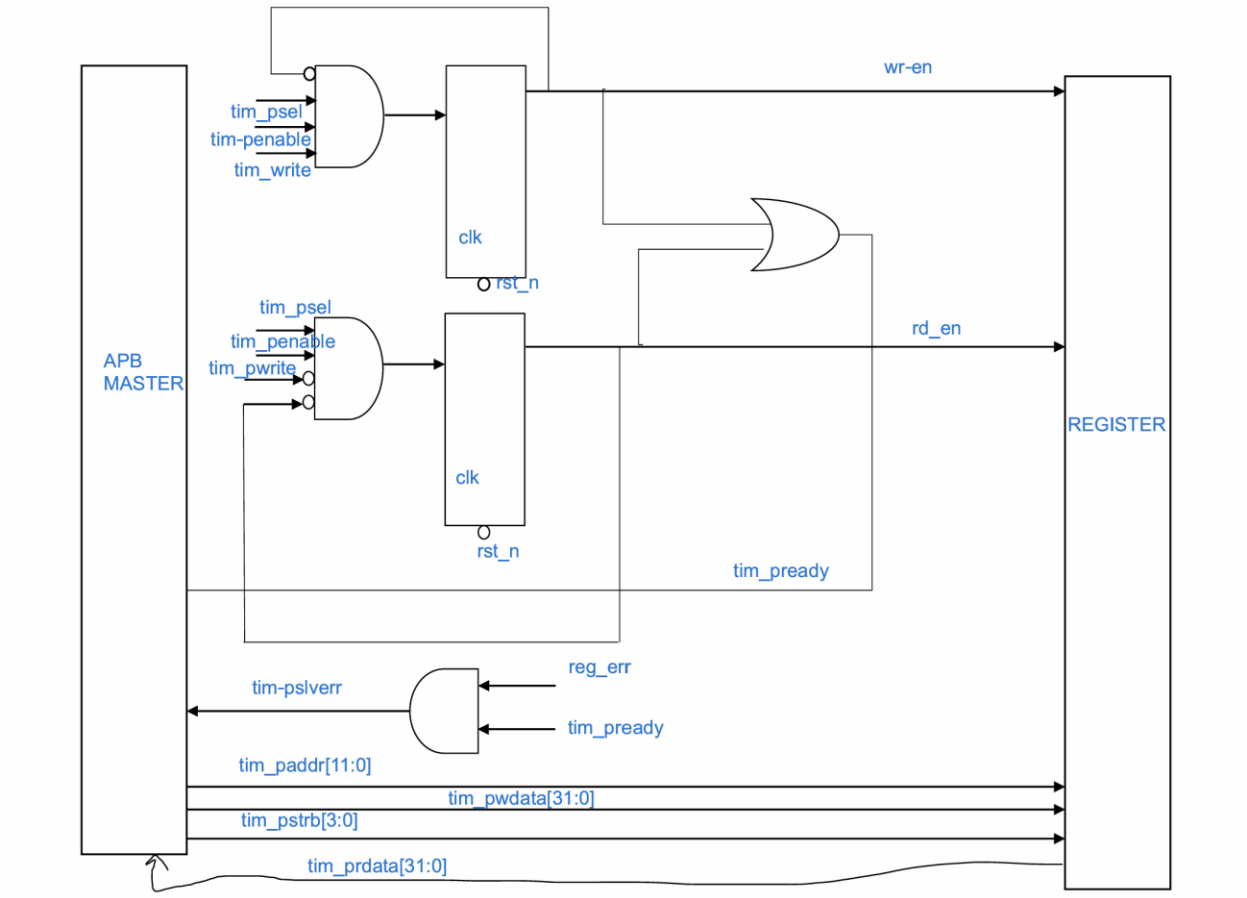


Figure 3.2: APB Slave Logic Diagram.

Logic Diagram Explanation: Figure 3.2 illustrates the gate-level implementation of the 1-cycle wait state. Combinational AND gates evaluate (tim_psel & tim_penable & tim_pwrite & ~wr_en) for writes and (tim_psel & tim_penable & ~tim_pwrite & ~rd_en) for reads. Feeding these into D Flip-Flops clocked by sys_clk introduces a 1-cycle delay and ensures wr_en and rd_en pulse High for exactly one clock cycle per transfer. The tim_pready output is generated by OR-ing wr_en and rd_en.

3.2. Register Module

The Register module serves as the central configuration and status hub of the Timer IP. It decodes 4-byte aligned addresses (addr[1:0] == 2'b00), controls byte-enable register updates, generates hardware error flags for prohibited TCR modifications, and multiplexes read data back to the APB interface.

3.2.1. Write Path and Specific Register Implementations

A. Timer Control Register (TCR) and Error Handling

The TCR (offset 12'h000) configures timer_en (bit 0), div_en (bit 1), and div_val[3:0] (bits 11:8). Because modifying the clock division settings while the timer is actively counting can corrupt timing intervals, hardware protection logic continuously monitors write requests targeting TCR (tcr_wr_sel = wr_en & (addr == 12'h000)).

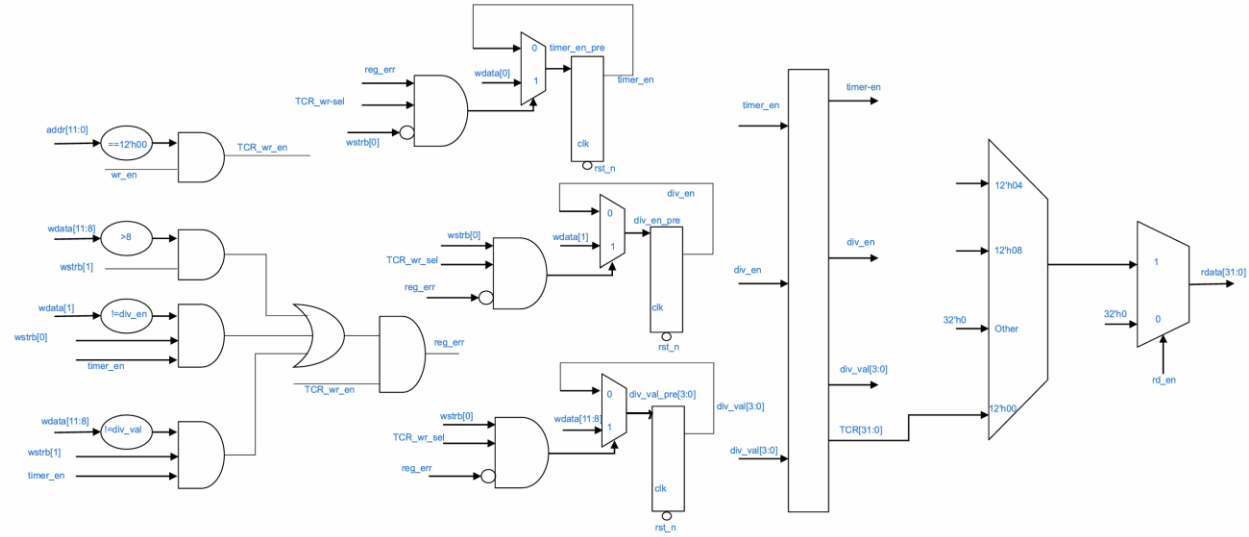


Figure 3.3: TCR Logic Diagram and Error Generation.

As depicted in Figure 3.3, the internal reg_err signal is asserted High if a write to TCR attempts any of the following:

- Writing a prohibited divisor value: tcr_wr_sel & wstrb[1] & (wdata[11:8] > 4'h8).
- Modifying div_val while timer_en is High: tcr_wr_sel & timer_en & wstrb[1] & (wdata[11:8] != div_val).
- Modifying div_en while timer_en is High: tcr_wr_sel & timer_en & wstrb[0] & (wdata[1] != div_en).

When reg_err is asserted, its inverted state (~reg_err) masks the write-select AND gates of all TCR fields (timer_en, div_en, and div_val), preserving their previous values and asserting tim_pslverr.

B. Timer Data Registers (TDR0, TDR1)

Unlike static configuration registers, TDR0 (12'h004) and TDR1 (12'h008) are dynamic counting registers physically residing inside the Counter module. The Register module decodes their addresses to generate tdr0_wr_sel and tdr1_wr_sel, forwarding them alongside wdata[31:0] and wstrb[3:0] to the Counter module.

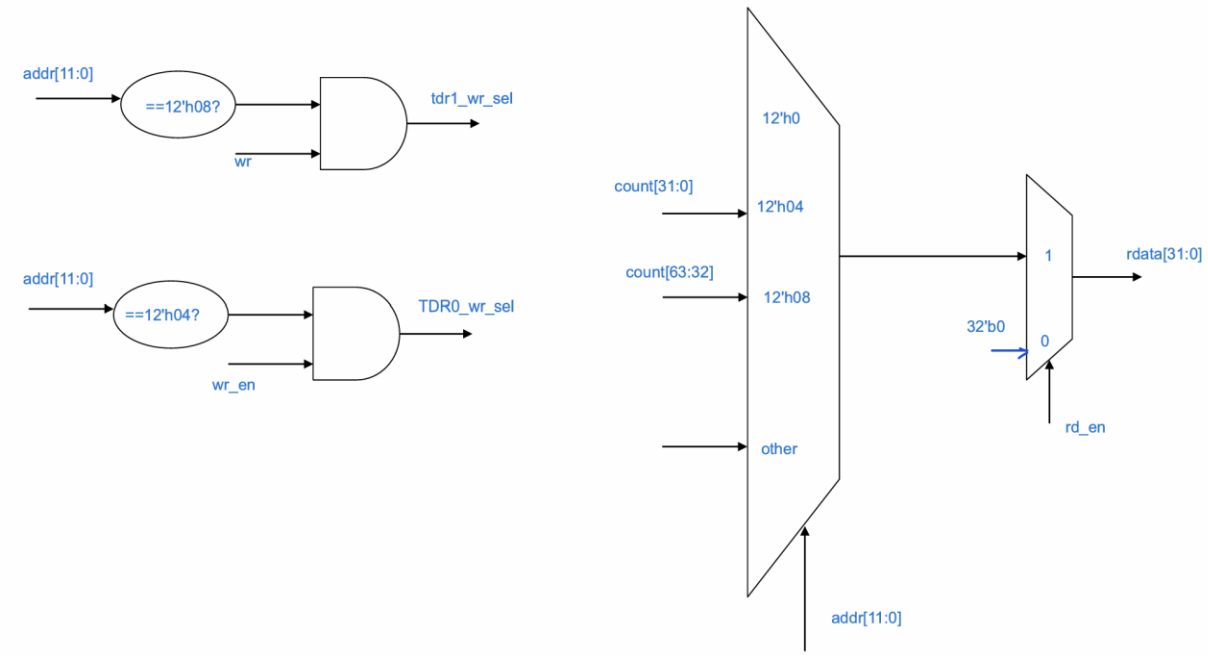


Figure 3.4: TDR0 and TDR1 Write Path Routing.

C. Timer Compare Registers (TCMP0, TCMP1)

TCMP0 (12'h00C) and TCMP1 (12'h010) store the 64-bit compare threshold (initialized to 32'hFFFF_FFFF on reset) and implement full byte-access write capability.

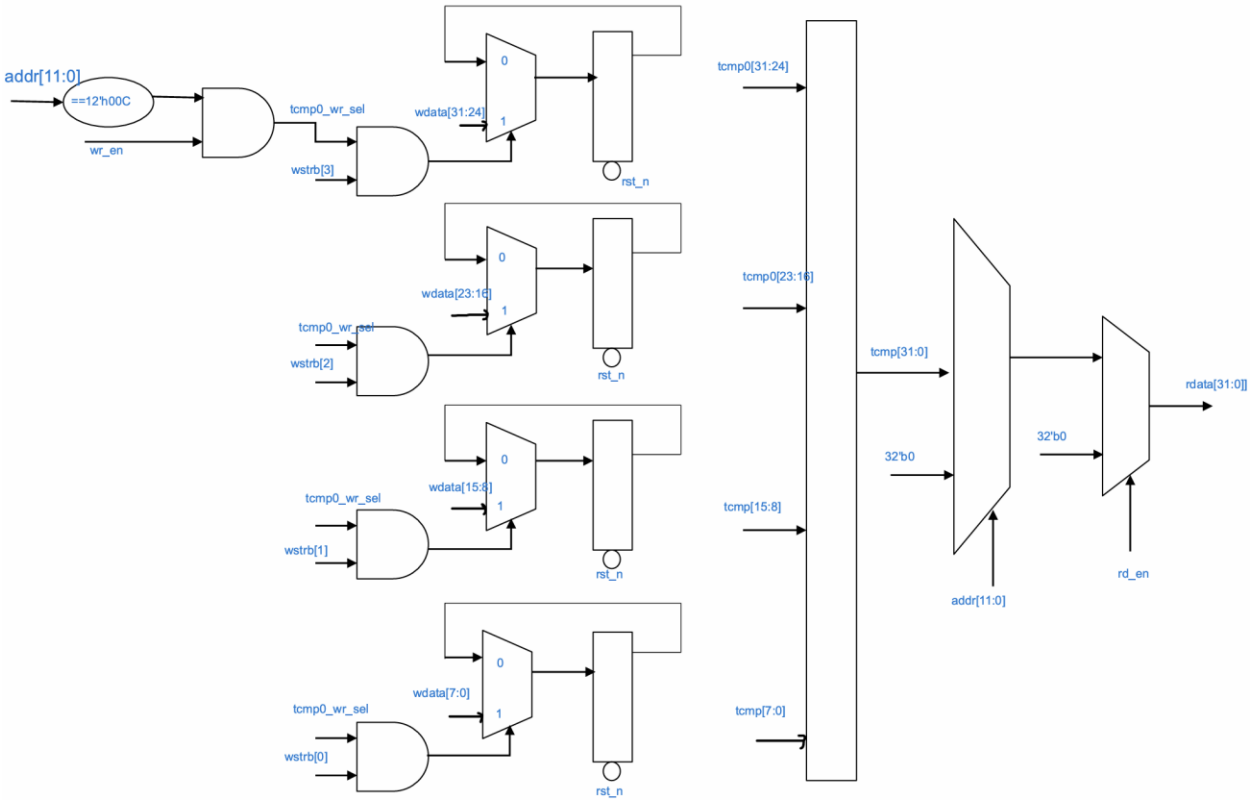


Figure 3.5: TCMP0 Register Logic Diagram Demonstrating Byte Access.

Figure 3.5 illustrates the byte-partitioned architecture of TCMP0 (identical for TCMP1). The 32-bit register is partitioned into four 8-bit D Flip-Flop banks. The register write select signal (tcmp0_wr_sel) is individually AND-ed with wstrb[3], wstrb[2], wstrb[1], and wstrb[0]. Only byte lanes with an active write strobe latch new data from wdata, while unselected byte lanes recirculate their current value (tcmp0[31:24], tcmp0[23:16], tcmp0[15:8], and tcmp0[7:0]).

D. Interrupt and Halt Registers (TIER, TISR, THCSR)

TIER (12'h014) and THCSR (12'h01C) use byte-enabled write logic (qualified by wstrb[0]) to update int_en (bit 0) and halt_req (bit 0), respectively. THCSR bit 1 (halt_ack) is a read-only status bit driven directly by the Counter Control module. TISR (12'h018) implements RW1C logic detailed in Section 3.5.

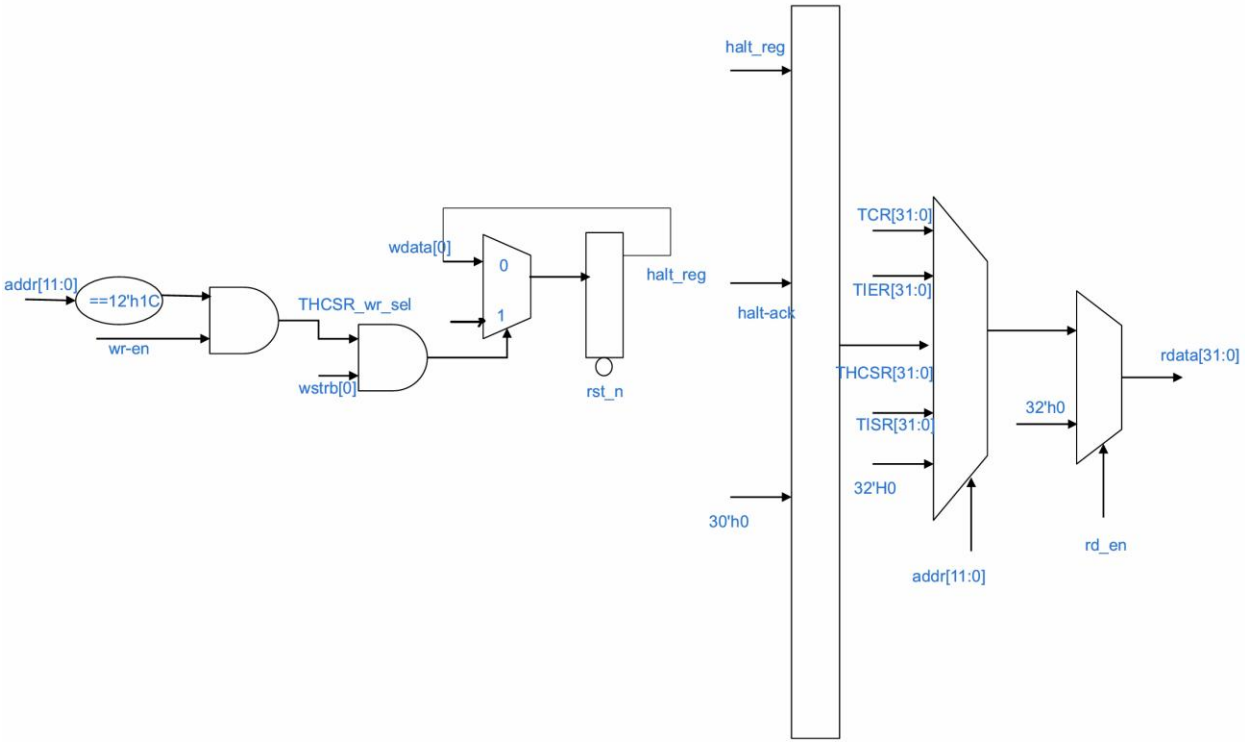


Figure 3.6: THCSR Register Logic Diagram.

3.2.2. Read Path and RAZ/WI Implementation

The read datapath uses a centralized Read Multiplexer controlled by addr[11:0]. Valid register offsets output their respective register values with all reserved bit positions hardwired to 0. Dynamic counter values for TDR0 (12'h004) and TDR1 (12'h008) are routed directly from count[31:0] and count[63:32]. Any unmapped or misaligned address selects the default 'Others' branch hardwired to 32'h0000_0000, ensuring strict RAZ/WI compliance. A final 2-to-1 MUX gated by rd_en drives tim_prdata to 32'h0000_0000 whenever no read transfer is active.

3.3. Counter Control Module

The Counter Control module generates the synchronous count enable pulse (cnt_en) for the 64-bit counter, manages the 8-bit prescaler (int_cnt[7:0]), enforces debug halt freezing, and detects falling edges on timer_en.

3.3.1. Logic Implementation

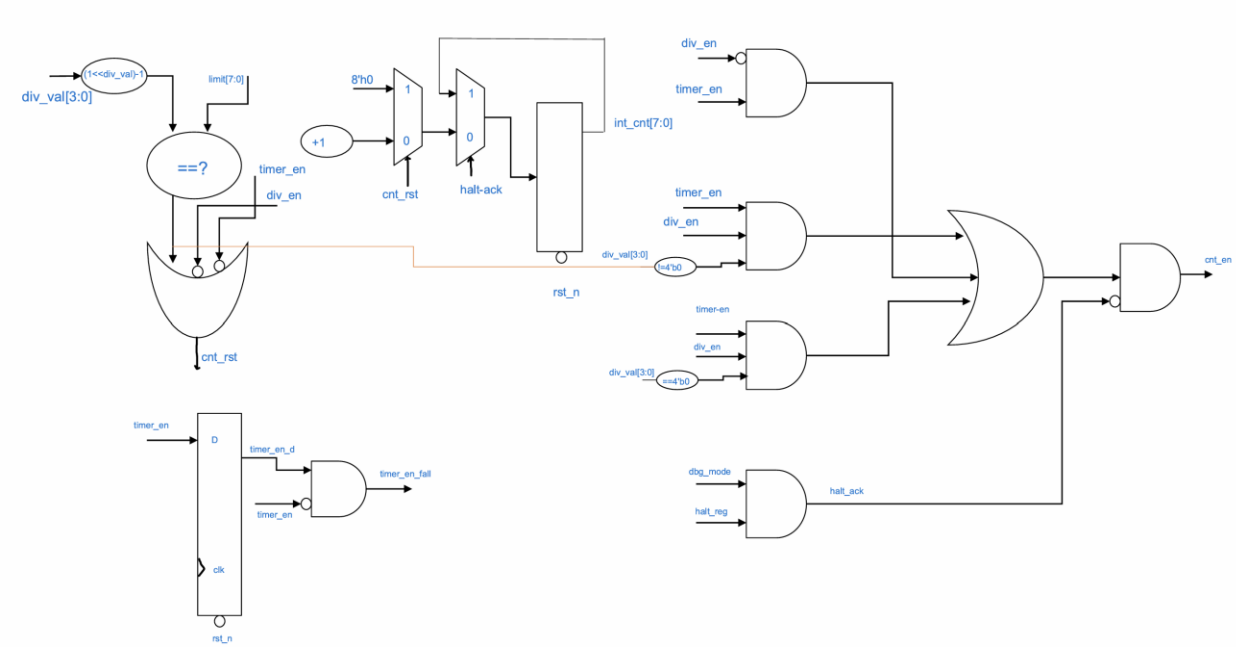


Figure 3.7: Counter Control Logic Diagram.

As shown in Figure 3.7, the module consists of five key functional sub-circuits:

- Dynamic Limit Calculation: Computes the terminal count for the prescaler using a left-shift operation: limit[7:0] = (8'd1 << div_val) - 8'd1.
- Prescaler Reset Logic (cnt_rst): Resets int_cnt[7:0] to 8'h00 when the timer is disabled (~timer_en), when division mode is disabled (~div_en), or when the terminal count is reached (int_cnt == limit): cnt_rst = (~timer_en) | (~div_en) | (int_cnt == limit).
- Preliminary Count Enable (count_en): Asserted High via a 3-input OR gate across three active counting conditions:
  (1) Default Mode: (~div_en) & timer_en
  (2) Control Mode (Un-divided, div_val == 0): div_en & timer_en & (div_val == 4'h0)
  (3) Control Mode (Divided, div_val != 0): div_en & timer_en & (div_val != 4'h0) & (int_cnt == limit)
- Halted Mode Enforcement: Asserts halt_ack = dbg_mode & halt_req. When halt_ack is High, the multiplexer immediately preceding the int_cnt[7:0] D-FF recirculates the

current int_cnt value, freezing the prescaler. Simultaneously, halt_ack masks the final enable output via an AND gate with an inverted input: cnt_en = count_en & (~halt_ack). Gating with ~halt_ack (rather than ~halt_req) guarantees that setting halt_req outside of debug mode (dbg_mode = 0) does not erroneously stop the counter.

- Falling Edge Detector (timer_en_fall): A D Flip-Flop delays timer_en by one clock cycle (timer_en_d). An AND gate evaluates timer_en_fall = timer_en_d & (~timer_en), generating a 1-cycle pulse whenever timer_en transitions from High to Low.

3.3.2. Timing and Behavior

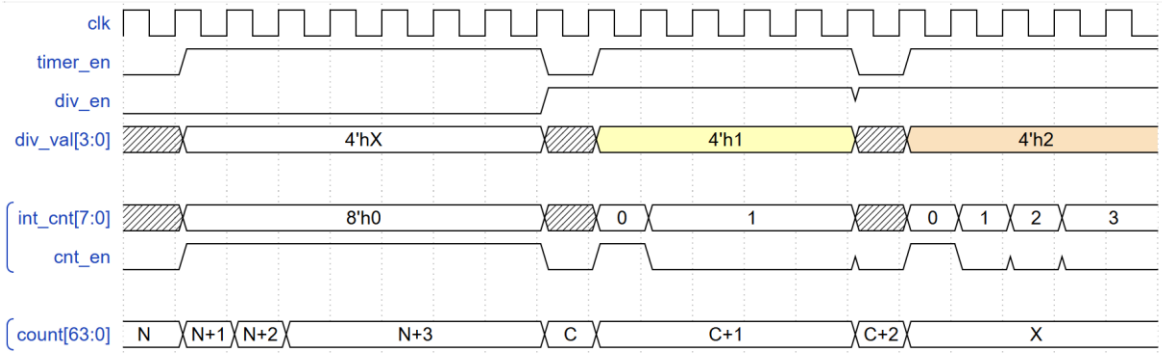


Figure 3.8: Counter Control Modes Timing Waveform.

Figure 3.8 provides a detailed waveform analysis demonstrating the dynamic behavior of the internal prescaler (int_cnt) across three distinct counting modes.

- **Default Mode:** When the control mode is disabled (div_en = 0), the timer operates at its maximum speed. The count enable signal (cnt_en) is driven High and remains continuously asserted as long as the timer is active (timer_en = 1). Consequently, the main 64-bit counter increments on every rising edge of the system clock.
- **Divide-by-2 Control Mode:** When the control mode is enabled (div_en = 1) with a divisor setting of 4'h1 (div_val = 1), the internal logic computes the terminal limit as 1 ((1 << 1) - 1). The 8-bit prescaler (int_cnt) continually toggles between 0 and 1. The cnt_en pulse is generated only when int_cnt reaches the limit of 1, effectively asserting the enable signal exactly once every 2 clock cycles and halving the main counter's speed.
- **Divide-by-4 Control Mode:** Maintaining div_en = 1 but increasing the divisor setting to 4'h2 (div_val = 2) results in a calculated limit of 3 ((1 << 2) - 1). In this configuration, the prescaler counts sequentially through the states 0, 1, 2, and 3 before resetting. The cnt_en pulse is asserted solely during the cycle where int_cnt equals 3, triggering an increment in the main counter exactly once every 4 clock cycles.

This mechanism ensures that the main 64-bit counting register remains fully synchronous with the system clock, utilizing enable pulses rather than gated clocks to achieve highly precise division ratios without introducing clock skew

3.4. Counter Module

The Counter module implements the 64-bit up-counter partitioned into TDR0[31:0] and TDR1[31:0]. It processes 64-bit increments, software byte-level presets, and hardware auto-clear upon timer disable.

3.4.1. Logic Implementation and Operation Priority

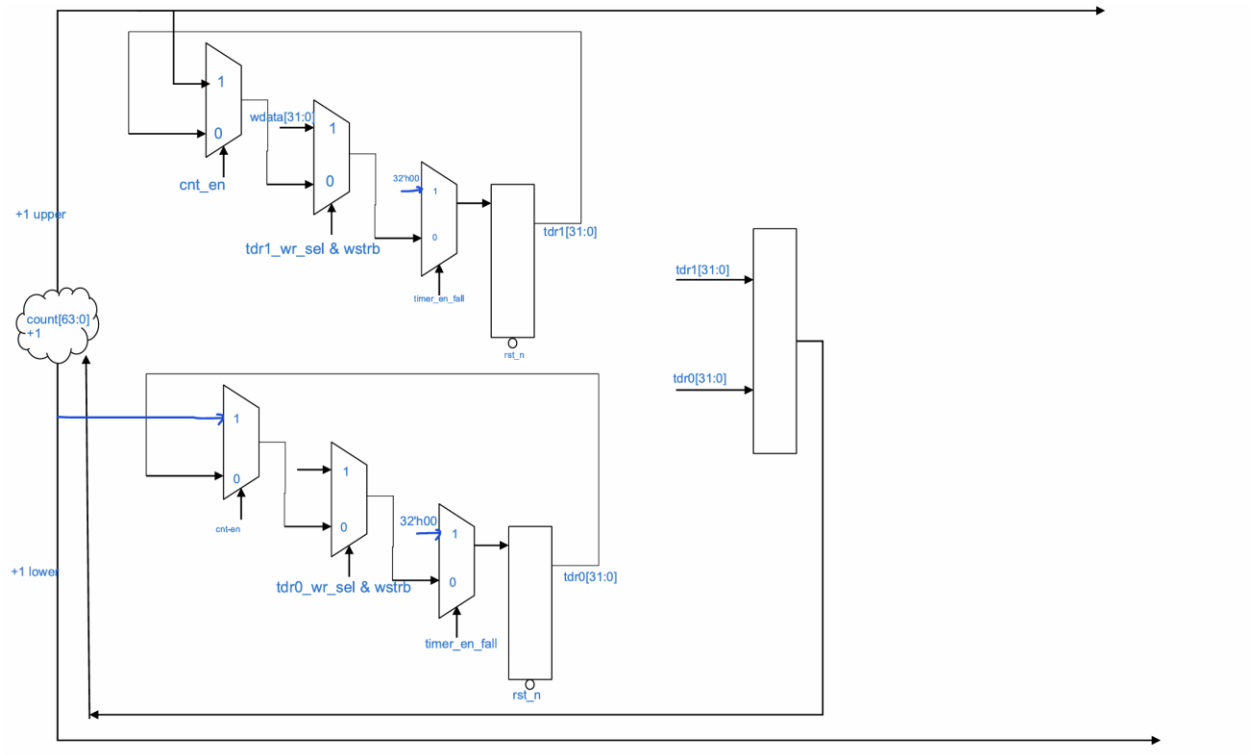


Figure 3.9: Counter Logic Diagram Illustrating Strict Prioritization.

As shown in Figure 3.9, cascaded multiplexers in front of the TDR0 and TDR1 D Flip-Flops enforce a strict three-level priority hierarchy:

- 1. Hardware Auto-Clear (Highest Priority): The final multiplexer stage directly feeding the D inputs of TDR0 and TDR1 is controlled by timer_en_fall. When timer_en transitions from High to Low (timer_en_fall = 1), both TDR0 and TDR1

synchronously load 32'h0000_0000, overriding any concurrent count increment or APB write.

- 2. APB Byte-Enable Write Preset (Medium Priority): The middle multiplexer stage is partitioned into four 8-bit byte lanes per 32-bit register. Each byte lane is controlled by (tdr0_wr_sel & wstrb[i]) for TDR0 and (tdr1_wr_sel & wstrb[i]) for TDR1 (where i = 0..3). Active byte lanes load wdata[(8*i+7):(8*i)] directly.
- 3. Counter Increment (Lowest Priority): The first multiplexer stage selects between the current 64-bit counter value (count[63:0]) and the incremented sum (count[63:0] + 64'd1) based on cnt_en.

3.4.2. Advanced Features Timing

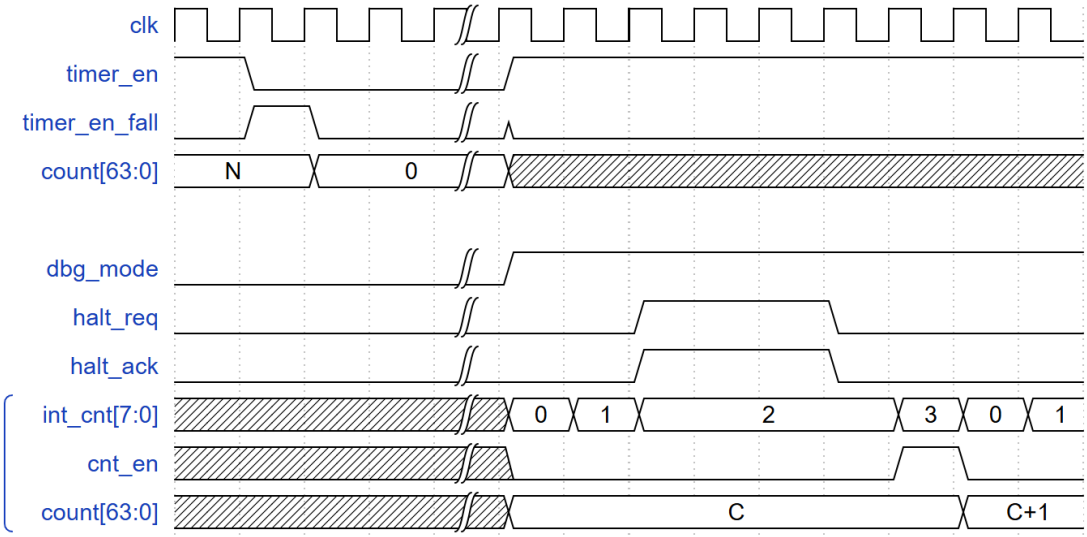


Figure 3.10: Hardware Auto-Clear and Halted Mode Timing Waveform.

Figure 3.10 illustrates both Advanced counter features. In the upper section, disabling timer_en generates a 1-cycle timer_en_fall pulse that clears count[63:0] to 0 on the next rising clock edge. In the lower section, asserting halt_req while dbg_mode = 1 immediately asserts halt_ack and freezes both int_cnt and count[63:0], resuming seamlessly from the paused state once halt_req is deasserted.

3.5. Interrupt Logic

The Interrupt module continuously compares the live 64-bit counter value against the 64-bit compare threshold and manages the RW1C interrupt status register (TISR.int_st) and maskable interrupt output (tim_int).

3.5.1. Logic Implementation and Clear Priority

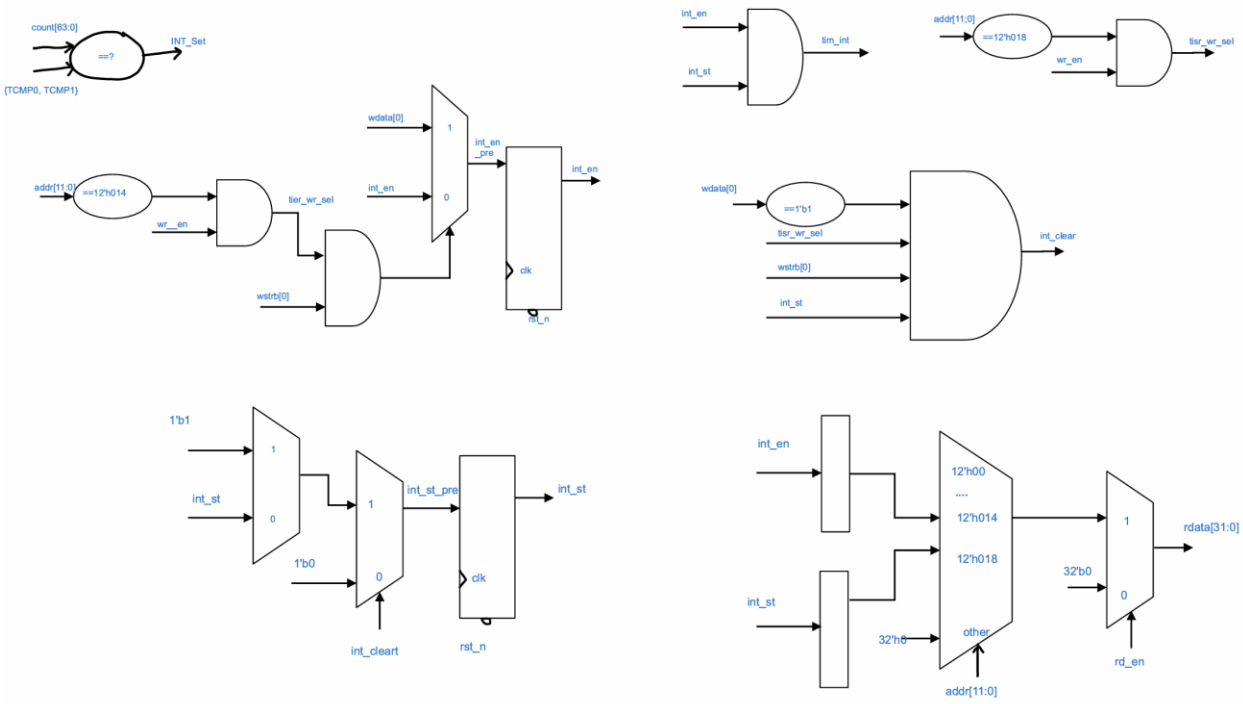


Figure 3.11: Interrupt Registers (TIER/TISR) Logic Diagram (Generation and RW1C Clearance).

As illustrated in Figure 3.11, the interrupt logic comprises four main mechanisms:

- Compare Match Detection (int_set): A 64-bit comparator asserts int_set = (count[63:0] == {TCMP1, TCMP0}) combinationally whenever the counter matches the compare registers.
- RW1C Software Clearance (int_clear): Decodes a write of 1 to bit 0 of TISR (offset 12'h018) when byte lane 0 is enabled: int_clear = tisr_wr_sel & wstrb[0] & wdata[0] & int_st.
- Clearance Priority Hierarchy: Per specification, software clearance has the highest priority. The multiplexer controlled by int_clear (selecting 1'b0) is placed closest to

the D input of the int_st Flip-Flop, overriding the preceding int_set multiplexer (selecting 1'b1) if both events occur simultaneously.

- Interrupt Masking: An AND gate combines the pending status and enable bit (tim_int = int_st & int_en), allowing software to mask tim_int without altering int_st.

3.5.2. Interrupt Lifecycle Timing

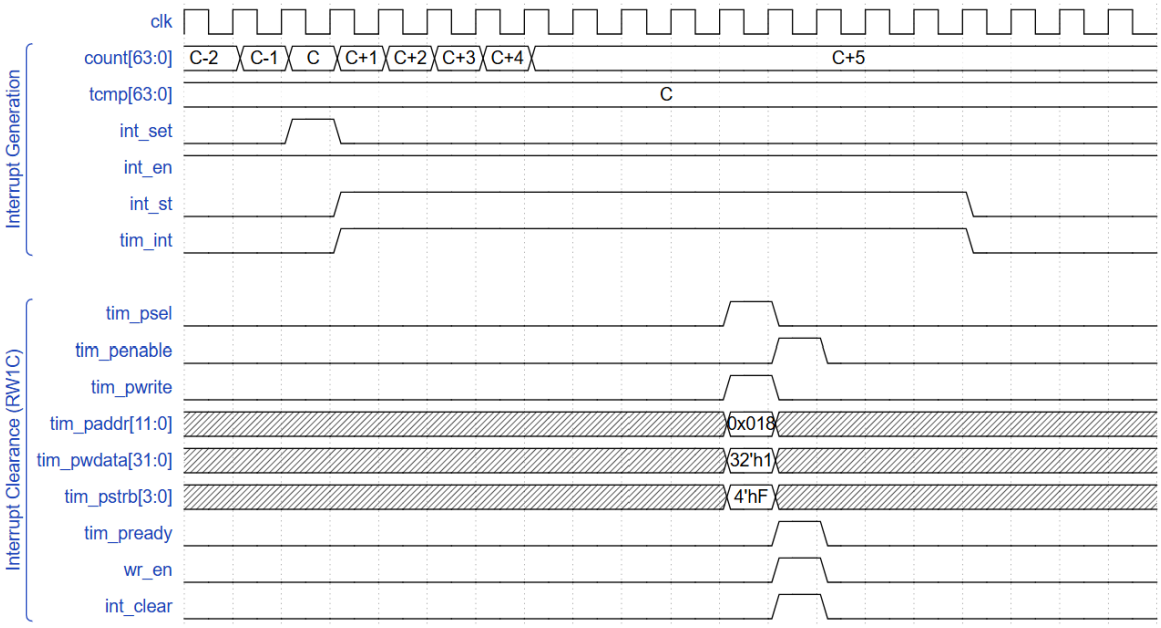


Figure 3.12: Interrupt Generation and RW1C Clearance Timing Waveform.

Figure 3.12 traces the complete interrupt lifecycle. When count[63:0] reaches the compare value C, int_set pulses High and latches int_st = 1 (and tim_int = 1 since int_en = 1) on the next rising clock edge, while the counter continues incrementing (C+1, C+2, ...). Later, the APB Master writes 32'h0000_0001 to TISR (0x018) with tim_pstrb[0] = 1. Following the 1-cycle APB wait state, wr_en and int_clear assert High, clearing both int_st and tim_int to 0 on the subsequent rising clock edge.

4. Revision History

Date	Version	Description	Author

Trinh Minh Quan